Experiment 8

Aim: To implement Part of Speech- tagging using HMM.

Code:

POS Tagging using HMM

Load and preprocess data

Load the Brown Corpus from NLTK and preprocess it by splitting it into training and testing sets and extracting words and POS tags.

```
import nltk
from nltk.corpus import brown import random

# nltk.download('brown') # Already downloaded

tagged_sentences = list(brown.tagged_sents()) random.seed(42)
random.shuffle(tagged_sentences)

split_ratio = 0.8
split_index = int(len(tagged_sentences) * split_ratio)

train_sentences = tagged_sentences[:split_index] test_sentences =
tagged_sentences[split_index:]

train_words = [] train_tags = []
for sentence in train_sentences: words, tags =
        zip(*sentence) train_words.extend(words)
        train_tags.extend(tags)

test_words = [] test_tags = []
for sentence in test_sentences: words, tags =
        zip(*sentence) test_words.extend(words)
        test_tags.extend(tags)

print(f"Number of training sentences: {len(train_sentences)}") print(f"Number of testing
sentences: {len(test_sentences)}") print(f"Number of training words: {len(train_words)}")
print(f"Number of training tags: {len(train_tags)}") print(f"Number of testing words:
{len(test_words)}") print(f"Number of testing tags: {len(test_tags)}")
```

```
Number of training sentences: 45872 Number of testing
sentences: 11468 Number of training words: 929015
```

```
Number of training tags: 929015
Number of testing words: 232177
Number of testing tags: 232177
```

Implement hmm

Implement the HMM from scratch, including functions for calculating emission and transition probabilities, and the Viterbi algorithm for decoding the most likely sequence of POS tags.

```
from collections import defaultdict

class HMM:
        def ____init_____(self, train_words,                    train_tags):

                self.train_words = train_words self.train_tags =
                train_tags
                self.tags = sorted(list(set(train_tags))) self.vocab =
                sorted(list(set(train_words))) self.num_tags = len(self.tags)
                self.num_vocab = len(self.vocab) self.tag_to_index = {tag: i for
                i, tag in
enumerate(self.tags)}
                self.word_to_index = {word: i for i, word in enumerate(self.vocab)}
                self.index_to_tag = {i: tag for i, tag in enumerate(self.tags)}

                self.transition_probabilities = self._calculate_transition_probabilities()
                self.emission_probabilities = self._calculate_emission_probabilities()

        def _calculate_transition_probabilities(self): transition_counts = defaultdict(lambda:
                defaultdict(int)) tag_counts = defaultdict(int)

                for i in range(len(self.train_tags) - 1): current_tag =
                        self.train_tags[i] next_tag = self.train_tags[i+1]
                        transition_counts[current_tag][next_tag] += 1
                        tag_counts[current_tag] += 1

                transition_probs = defaultdict(lambda: defaultdict(float)) for current_tag, next_tags in
                transition_counts.items():
                        for next_tag, count in next_tags.items(): transition_probs[current_tag][next_tag] = count /
tag_counts[current_tag]

                return transition_probs
```

```
    def _calculate_emission_probabilities(self): emission_counts = defaultdict(lambda:
        defaultdict(int)) tag_counts = defaultdict(int)

        for word, tag in zip(self.train_words, self.train_tags): emission_counts[tag][word] += 1
            tag_counts[tag] += 1

        emission_probs = defaultdict(lambda: defaultdict(float)) # Add a small smoothing
        factor for unknown words/tags smoothing_factor = 1e-5
        for tag, words in emission_counts.items(): for word, count in
            words.items():
                emission_probs[tag][word] = (count + smoothing_factor)
/ (tag_counts[tag] + smoothing_factor * self.num_vocab)

        # Handle words not seen with a specific tag during training
        for tag in self.tags:
            if tag not in emission_probs: emission_probs[tag] = defaultdict(float)
            for word in self.vocab:
                if word not in emission_probs[tag]: emission_probs[tag][word] = smoothing_factor /
(tag_counts.get(tag, 0) + smoothing_factor * self.num_vocab)


        return emission_probs

    def viterbi_decode(self, sequence):
        sequence = [word.lower() for word in sequence] # Convert to lowercase for better matching
        T = len(sequence)
        delta = {} # Stores the maximum probability of being in state i at time t
        psi = {} # Stores the argument that maximizes the probability
in delta

        # Initialization (t=0) first_word =
        sequence[0] for tag in self.tags:
            # Consider transition from a start state implicitly (e.g., uniform probability or based on
sentence start tags if available)
            # Here, we'll assume a uniform probability for the first tag for simplicity, combined
with the emission probability
            initial_transition_prob = 1.0 / self.num_tags # Simplified initial probability
            emission_prob = self.emission_probabilities.get(tag,
{}).get(first_word, 1e-10) # Add small value for unseen words delta[(0, tag)] = initial_transition_prob *
            emission_prob psi[(0, tag)] = None # No preceding state for the first
```

```
element

            # Recursion (t=1 to T-1)
            for t in range(1, T): current_word = sequence[t]
                for current_tag in self.tags:
                    max_prob = -1 best_prev_tag = None
                    for prev_tag in self.tags: transition_prob =
self.transition_probabilities.get(prev_tag, {}).get(current_tag, 1e-10) # Add small value for unseen
transitions
                        emission_prob = self.emission_probabilities.get(current_tag,
{}).get(current_word, 1e-10) # Add small value for unseen words
                        prob = delta[(t-1, prev_tag)] * transition_prob *
emission_prob
                        if prob > max_prob: max_prob = prob
                            best_prev_tag = prev_tag delta[(t, current_tag)]
                    = max_prob psi[(t, current_tag)] = best_prev_tag

            # Termination
            max_prob_T = -1
            last_tag = None
            for tag in self.tags:
                if delta[(T-1, tag)] > max_prob_T: max_prob_T =
                    delta[(T-1, tag)] last_tag = tag

            # Path Backtracking best_tag_sequence = [None]
            * T best_tag_sequence[T-1] = last_tag for t in
            range(T-2, -1, -1):
                best_tag_sequence[t] = psi[(t+1, best_tag_sequence[t+1])] return best_tag_sequence
```

Train hmm

Train the HMM using the training data by calculating the emission and transition probabilities from the corpus.

```
hmm_model = HMM(train_words, train_tags)
```

Evaluate hmm

Evaluate the trained HMM on the test data by comparing the predicted POS tags to the actual POS tags and calculating the accuracy.

```
predicted_tags = []
# Use a subset of the test sentences for faster evaluation subset_size = 10   # You can adjust this
number
for sentence in test_sentences[:subset_size]: words = [word for word,
        tag in sentence]
        predicted_sentence_tags = hmm_model.viterbi_decode(words)
        predicted_tags.extend(predicted_sentence_tags)

# Collect the corresponding actual tags for the subset
actual_tags_subset = []
for sentence in test_sentences[:subset_size]: tags = [tag for word, tag
        in sentence] actual_tags_subset.extend(tags)


correct_predictions = sum(1 for pred, actual in zip(predicted_tags, actual_tags_subset) if pred == actual)
total_tags = len(actual_tags_subset) accuracy =
correct_predictions / total_tags

print(f'Accuracy of the HMM on a subset of the test set ({subset_size} sentences): {accuracy:.4f}")
```

```
Accuracy of the HMM on a subset of the test set (10 sentences): 0.9198
```

Predict POS tags

Use the trained HMM to predict POS tags for sample sentences.

```
sample_sentences = [
    "The quick brown fox jumps over the lazy dog .",
    "NLTK is a powerful library for natural language processing .",
    "Hidden Markov Models are used in many applications ."
]

print("Predicting POS tags for sample sentences:")
for sentence in sample_sentences:
    words = sentence.split()
    predicted_tags = hmm_model.viterbi_decode(words)
    print(f"Sentence:  {sentence}")
    print(f"Predicted tags: {predicted_tags}")
    print("-" * 20)
```

```
Predicting POS tags for sample sentences:
Sentence: The quick brown fox jumps over the lazy dog .
Predicted tags: ['AT', 'JJ', 'JJ', 'NN', 'VBZ', 'IN', 'AT', 'JJ',
'NN', '.']
----------------------
Sentence: NLTK is a powerful library for natural language processing .
Predicted tags: ['NR-NC', 'BEZ', 'AT', 'JJ', 'NN', 'IN', 'JJ', 'NN',
'NN', '.']
----------------------
Sentence: Hidden Markov Models are used in many applications .
Predicted tags: ['VBN', 'IN', 'NNS', 'BER', 'VBN', 'IN', 'AP', 'NNS',
'.']
----------------------
```

Visualize probabilities

Visualize the emission and transition probabilities.

```
import matplotlib.pyplot as plt import seaborn as
sns
import pandas as pd

# Visualize Emission Probabilities (subset for clarity) emission_probs_df =
pd.DataFrame(hmm_model.emission_probabilities).T.fillna(0)

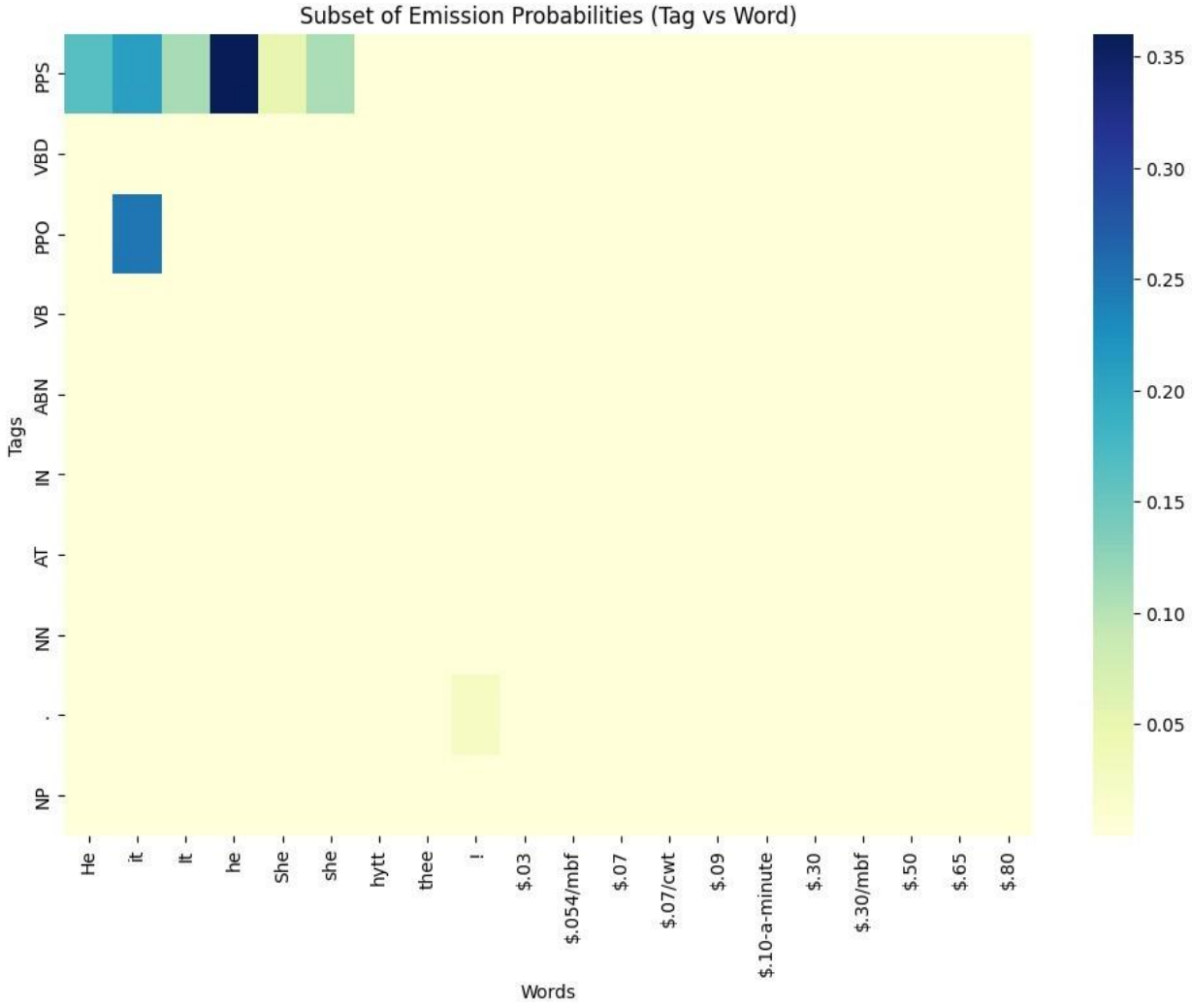
# Select a subset of tags and words for visualization
subset_tags = list(emission_probs_df.index)[:10] # Visualize first 10 tags
subset_words = list(emission_probs_df.columns)[:20] # Visualize first
20 words

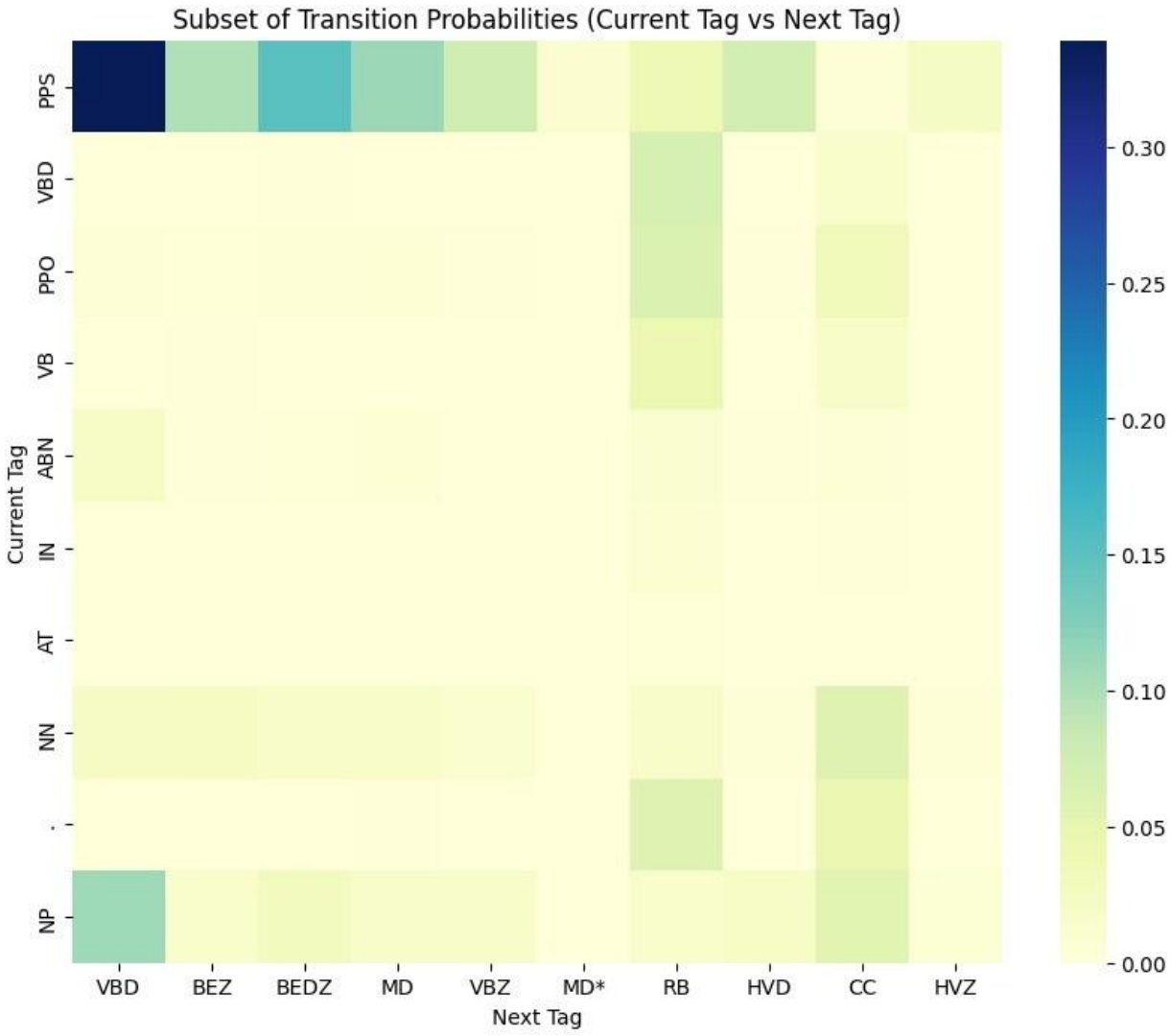
plt.figure(figsize=(12, 8)) sns.heatmap(emission_probs_df.loc[subset_tags, subset_words],
cmap="YlGnBu", annot=False)
plt.title("Subset of Emission Probabilities (Tag vs Word)") plt.xlabel("Words")
plt.ylabel("Tags") plt.show()

# Visualize Transition Probabilities (subset for clarity) transition_probs_df =
pd.DataFrame(hmm_model.transition_probabilities).T.fillna(0)

# Select a subset of tags for visualization subset_tags_transition =
list(transition_probs_df.index)[:10] #
Visualize transitions from first 10 tags
subset_next_tags_transition = list(transition_probs_df.columns)[:10] # Visualize transitions to first 10
tags
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(transition_probs_df.loc[subset_tags_transition,
subset_next_tags_transition], cmap="YlGnBu", annot=False)
plt.title("Subset of Transition Probabilities (Current Tag vs Next
Tag)")
plt.xlabel("Next Tag")
plt.ylabel("Current Tag")
plt.show()
```




Subset of Transition Probabilities (Current Tag vs Next Tag)
Current Tag
PPS
VBD
PPO
VB
ABN
IN
AT
NN
.
NP
VBD
BEZ
BEDZ
MD
VBZ
MD*
RB
HVD
CC
HVZ
Next Tag
0.30
0.25
0.20
0.15
0.10
0.05
0.00